TEMCO LEARN

Temco Platform

`temco-store` · built Nov 10, 2026

`temco-store`

The platform's own memory: one SQLite file per window, the selection, and the naming rules the tables are keyed by. It is the base every other module stands on.

Folder	`extensions/temco-store`
Extension id	`temcocontrols.temco-store`
Namespace	`temcoStore`
State	active
Surface	one command (`temcoStore.status`) — no view, no container, no settings
Requires	nothing
Required by	`temco-docs` (and, through it, `temco-ai`); `temco-repositories` and `temco-projects` when they are split out

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, and the API below
`src/api.ts`	the published contract: `available()`, `catalog()`, `CatalogUnavailable`, `selection`, `slugOf`
`src/catalog/schema.ts`	the tables, one per idea, and `SCHEMA_VERSION` — a bump is a migration, and a newer database is refused in words
`src/catalog/store.ts`	`node:sqlite`: the open, the pragmas, the schema, and every statement the platform runs — sources, projects (`documents` in the tables), their sources, recipes, builds, artifacts, commits, pages, scans, events
`src/selection.ts`	which project the window is on and the page it last opened from it, in `workspaceState`, with the one signal the views and the AI listen to
`src/names.ts`	`slugOf` — the name → key rule the tables assume
`src/commands.ts`	the one command: what the store is, where it is, what it holds

The catalog, in one paragraph

A **source** is somewhere documentation was read from: a repository, a folder, an archive, with the branch and the local copy when there is one. A **project** (the table is still `documents`; renaming it buys a migration and nothing else) is a name, a version, an audience, an owner, the sources it is made of, and the outputs it produces. A **read** records what a scan found — the page ids, their paths, their owners — and what it concluded, so a list can say "read, 42 pages" without re-reading anything. **Builds** and **artifacts** are what the project's own pipeline produced, keyed by path so a second build of the same site does not turn one output into two rows. **Events** are what happened, and `unseenEvents()` is how a surface says "something moved". **Commits** are the history a draft is about, per source and per page.

Boundary rules

- **It has no domain opinions.** It stores and returns rows; it does not decide what a project is, what to read, or what to build. Those are `temco-projects`' and `temco-repositories`' questions.
- **It knows no view.** No webview, no tree, no icon: the command prints to an output channel and that is all.
- **It is not `temco-sql`.** That module is about the databases a *product* uses (registers, queries, export). This is the platform's own file, and the reserved module keeps its meaning.
- **It is never a repository's storage.** Nothing is written into a product repository: the database lives in this module's `globalStorageUri` (`<storage>/catalog/catalog.db`) and the materialised trees in a `sources/` cache beside it.

- **A caller passes no context.** `Catalog.open(context)` exists for compatibility at the call sites, and the argument is ignored — the file is the store's, opened with the store's own storage.

Callers

A consumer mirrors what it needs in its own `src/store.ts` (an extension cannot import another extension's sources) and declares this module in `extensionDependencies`, so the exports exist before its own `activate()` runs. `extensions/temco-docs/src/store.ts` is the copy to follow: it re-presents the same names — `Catalog`, `CatalogUnavailable`, `Selection`, `slugOf` — so a call site reads as it did before the split.